

2025 年《 计算机操作系统 》
课程设计指导手册
(面向计算机科学与技术专业)
第 V1 版

南京农业大学人工智能学院
姜海燕

jianghy@njau.edu.cn

2025 年 2 月 14 日

目 录

1 课程设计目的与要求	1
1.1 实验目的	1
2.2 选题及要求	1
名称：仿真实现操作系统的作业管理及内存管理系统	1
2.2.1 基础要求（针对所有成绩等级）	1
2.2.2 及格要求（成绩等级：D）	2
2.2.3 中等要求（成绩等级：C）	3
2.2.4 良好要求（成绩等级：B 系列，包括 B、B+）	3
2.2.5 优秀要求（成绩等级：A 系列，包括 A、A+）	4
2.基础的仿真设计	5
2.1 裸机硬件仿真设计	5
(1) CPU 与寄存器的仿真设计	5
(2) 系统时钟中断仿真	5
(3) 作业请求的中断仿真	6
(4) 进程调度的中断仿真	6
(5) 用户内存区	7
(6) 进程的磁盘数据输入缓冲区	7
(7) 输入/输出中断事件仿真	7
(8) 打印作业的预输入缓冲区、缓输出缓冲区及打印井	错误！未定义书签。
2.2 并发作业请求文件（jobs-input.txt）的设计	7
2.4 作业运行及调度详细记录的文件保存	9
2.6 进程控制块 PCB 设计	11
2.7 系统 PCB 表的设计	错误！未定义书签。
2.8 仿真实现进程控制原语	13
3 2024-2025 年第一学期课程设计时间节点安排	12
3.1 第 1 周，阅读课程设计指导手册，周三选题动员（地点另行通知），学生申请成绩等级	12
3.2 第 2 周，周三晚上 6:00-7:00，课程设计线上辅导讲解，学生编写课设代码	12
3.4 第 3 周，周三晚上课设线上答疑，学生编写课设代码	12
3.5 第 4 周，周三晚上课设线上答疑，学生修改课设代码并测试	12
3.6 第 5 周，3 月 25 日-29 日，全周停课课设, 地点：滨江校区实验室（地点另行通知）	12
3.7 正式提交全部材料时间：	12
3 月 28 日（周四）提交自己的课设完整材料；	12
3 月 29 日（周五）提交互测完整材料	12
3.8 停课 5 天时间安排：	12
4 自测材料提交要求	12
5 课程设计课程联系方式	13

1 课程设计目的与要求

1.1 课设目的

以计算机操作系统原理为指导，利用多线程并发编程以及面向对象程序设计技术仿真 OS 内核的进程管理、作业管理、连续内存管理等 API 函数功能，可视化显示操作系统作业管理及进程管理的工作过程，培养训练学生独立开发、调试等工程能力。本次课程设计需要按本手册要求完成基础实验、所申请成绩等级对应的仿真程序，使用教师发布的基础实验代码框架、课设基础代码框架上扩展编程，使用教师提供的测试数据按照测试模版要求测试，撰写自测互测报告，录制代码设计实现的讲解视频、自测、互测过程讲解视频等文件。

1.2 选题及要求

名称：仿真实现操作系统的作业管理及内存管理系统

1.2.1 基础要求（针对所有成绩等级）

◆ 本课设分为三个阶段。

第一阶段：基础实验及课设编程

第 1-2 周，完成 2 个基础实验，并独立运行测试；

第 3-4 周，每位同学独立完成申请成绩等级对应的课程设计任务，使用教师发布的课设基础代码框架扩展编程，使用教师提供的测试数据测试

第二阶段：测试与撰写报告

第 5 周（停课 5 天），按照测试模版要求测试、修改程序，撰写测试分析报告，录制代码实现原理及测试过程讲解视频文件。完成学生之间互测评价。

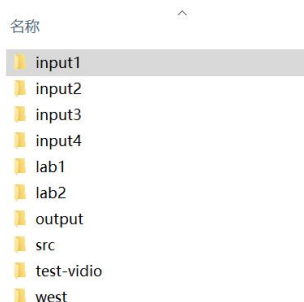
第三阶段：教师测试与评价（第 6-13 周）

◆ 课程设计采用的语言为 JAVA 或 C#，需要提交开发环境下运行代码以及脱离开发环境独立测试运行的可执行程序。使用 IDL 开发环境

◆ 根据本手册要求和自己的实践能力，选择申请的成绩等级和需要实现的功能，详细设计要求阅读后续小节。跨级实现不得分。成绩申请等级分别为：A、B、C、D，不提交课设材料的成绩视为不合格。

◆ 基础实验必须使用教师发布的 lab1、lab2 文件夹下提供的程序代码框架、测试数据完成基础实验，可执行程序需要按照学习通发布的作业完成基础实验的测试与互评；

◆ 每位同学创建“姓名学号-申请成绩”文件夹，拷贝教师发布的课设材料模版中已创建好文件夹作为子文件夹。教师发布的课设材料模版在第三周发布。



（1）该文件夹的根目录：保存课设程序被编译为 RunProcess.exe 可执行文件、自测

分析报告（模版在第 5 周停课课设第 1 天发布），提交材料阅读顺序及注意事项。课设材料安装使用说明书等

(2) 必须使用 src 文件夹下教师发布的课设基础代码框架上进行扩展编程，根据设计要求自行添加变量、属性、方法等代码，并补充注释。src 文件夹下的程序需在开发环境下可运行。与申请成绩等级及功能不匹配的代码模块和程序文件请不要出现在该文件夹，否则视为抄袭嫌疑。

(3) 必须使用 input1 至 input4 文件夹下成绩等级对应的测试数据进行测试分析，调试运行输出结果保存到 output 文件夹中。申请不同成绩等级的同学请认真阅读该手册，对输入测试数据格式要求不同。不使用教师提供测试数据进行测试，或者使用往年测试数据及格式，本课程成绩视为不合格。

(4) test-vidio 子文件夹:根据测试分析模板要求，按要求保存多个用例过程的录屏讲解文件（可以多个）。单个文件大小最好不超过 30M;

(5) west 子文件夹: 安装该程序需要的配套环境

(6) lab1、lab2 子文件夹: 保存所完成的基础性实验完整代码、可执行程序、测试数据与结果

1.2.2 及格要求（成绩等级：D）

(1) 利用 2 个线程仿真实现时钟中断及作业调度、进程调度，实现代码分别写在 ClockInterruptHandlerThread、ProcessSchedulingHandlerThread 文件中，并补充完整；

(2) 多任务作业并发环境下，实现时钟计数、作业请求判别及调度、进程创建、进程撤销、进程调度等原语、采用静态优先级+时间片轮转进程调度算法，实现进程就绪态与运行态之间的调度切换；

(3) 不考虑内存分配，创建新进程时不受内存大小限制；

(4) 界面上需要设计 4 个按钮，分别为执行、暂停、保存、实时。

“执行”按钮：按下时系统时钟开始计数并显示，作业调度、进程调度开始工作；

“暂停”按钮：按下时，系统时钟暂时中断、暂停作业调度、进程调度功能；

“保存”按钮：按下时将作业调度、进程调度过程的记录信息保存到 output 文件夹中；

“实时”按钮：按下以后当前时刻有实时作业请求，实现实时作业指令生成、作业调度、进程调度、进程撤销等操作；实时作业指令包括 20 条，每条指令均为计算类型。

(5) 界面上划分 4 个区域，分别为时钟显示区、作业请求区、进程就绪区、进程运行区。

时钟显示区：位置在左上角，区域较小，字体为黑色，当仿真程序一旦运行，时钟显示区开始显示按秒计时。

作业请求区：位置在左中下，用行形式，每行显示一个作业全部 JCB 信息。

进程就绪区：位置在右中上，用行形式显示就绪队列进程 PCB 信息，每行信息均需要带时间、作业编号、进程编号、指令编号等；

进程运行区：位置在右下角，显示正处于运行态进程的 PCB 信息，要带时间、作业编号、进程编号、指令编号等；

当按下界面按钮对作业和进程开始调度，作业请求区、进程就绪区、进程运行区动态显示作业和进程调度过程信息；

(6) 使用 input4 文件夹读取每个指令文件信息；调度运行过程、JCB、PCB 信息变化过程记录保存到 output 文件夹；

1.2.3 中等要求（成绩等级：C）

（1）利用 [2 个线程仿真](#) 实现时钟中断、作业并发调度、进程调度请求，实现代码分别写在 ClockInterruptHandlerThread、ProcessSchedulingHandlerThread 文件中，请补充完整；

（2）[仿真可变分区内存管理，考虑 A、B 两类外部设备，采用静态方式分配外部设备，忽略用户作业的 I/O 操作时间。](#)实现 MMU 地址变换，利用最佳适应算法实现可变分区内存空间分配与回收，注意要考虑内存、外设情况进行作业调度；

（3）多任务作业并发环境下，仿真实现时钟计数、作业请求判别及调度、系统调用、进程创建、进程撤销、进程调度、内存分配、内存回收等原语；采用[静态优先级+时间片轮转进程调度算法，实现进程就绪态、运行态之间的调度切换；](#)

（4）[界面上需要设计 4 个按钮，分别为执行、暂停、保存、实时。](#)

“执行”按钮：按下时系统时钟开始计数并显示，作业调度、进程调度开始工作；

“暂停”按钮：按下时，系统时钟暂时中断、暂停作业调度、进程调度功能；

“保存”按钮：按下时将作业调度、进程调度过程的记录信息保存到 output 文件夹中；

“实时”按钮：按下以后当前时刻有[实时作业请求](#)，实现实时作业指令生成、作业调度、进程调度、进程撤销等操作；实时作业指令包括 20 条，每条指令均为计算类型。

（5）[界面上划分 5 个区域，分别为时钟显示区、作业请求区、内存区、进程就绪区、进程运行区。](#)

时钟显示区：位置在左上角，区域较小，字体为黑色，当仿真程序一旦运行，时钟显示区开始显示按秒计时。

作业请求区：位置在左中下，用行形式，每行显示一个作业全部 JCB 信息。

进程就绪区：位置在右中上，用行形式显示就绪队列进程 PCB 信息，每行信息均需要带时间、作业编号、进程编号、指令编号等；

进程运行区：位置在右下角，显示正处于运行态进程的 PCB 信息，要带时间、作业编号、进程编号、指令编号等；

内存区：位置在中部，设计[二维表的图形化界面](#)可视化呈现内存分配、撤销时的内存管理过程；分配的内存区域用红色，表示出开始地址、结束地址。

当按下界面按钮对作业和进程开始调度，作业请求区、进程就绪区、进程运行区动态显示作业和进程调度过程信息；

（6）使用 [input3 文件夹读取每个指令文件信息](#)；调度运行过程、JCB、PCB 信息变化过程记录保存到 output 文件夹；

1.2.4 良好要求（成绩等级：B 系列，包括 B、B+）

（1）利用 [4 个线程仿真](#) 实现时钟中断及作业并发调度、进程调度、键盘输入及屏幕显示输出中断请求；时钟中断及作业并发调度、进程调度实现代码框架分别写在 ClockInterruptHandlerThread、ProcessSchedulingHandlerThread 文件中，请补充完善；[键盘输入及屏幕显示输出中断请求，分别使用 2 个单独的新线程程序文件，需要自行设计完成；](#)

（2）[仿真可变分区内存管理，考虑 A、B 两类外部设备，采用静态方式分配外部设备，忽略用户作业的 I/O 操作时间。](#)实现 MMU 地址变换，利用最佳适应算法实现可变分区内存空间分配与回收，注意要考虑内存、外设情况进行作业调度；

（3）多任务作业并发环境下，仿真实现时钟计数、作业请求判别及调度、系统调用、进程创建、进程撤销、进程调度、进程阻塞、进程唤醒、内存分配、内存回收等原语；[采用三级反馈队列进程调度算法实现进程就绪态、运行态、阻塞态之间进程切换。](#)不考虑使用内

存缓冲区；

(4) 界面上需要设计 4 个按钮，分别为执行、暂停、保存、实时。

“执行”按钮：按下时系统时钟开始计数并显示，作业调度、进程调度开始工作；

“暂停”按钮：按下时，系统时钟暂时中断、暂停作业调度、进程调度功能；

“保存”按钮：按下时将作业调度、进程调度过程的记录信息保存到 output 文件夹中；

“实时”按钮：按下以后当前时刻有实时作业请求，实现实时作业指令生成、作业调度、进程调度、进程撤销等操作；实时作业指令包括 20 条，其中 15 条计算类、3 条键盘输入指令、2 条屏幕显示输出指令，指令出现顺序自行设计规则。

(5) 界面上划分 6 个区域，分别为时钟显示区、作业请求区、内存区、进程就绪区、进程运行区、进程阻塞区。

时钟显示区：位置在左上角，区域较小，字体为黑色，当仿真程序一旦运行，时钟显示区开始显示按秒计时。

作业请求区：位置在左中，用行形式，每行显示一个作业全部 JCB 信息。

进程就绪区：位置在右中上，分三级区域显示。每级对应三级反馈队列的一级。每一级用行形式显示就绪队列进程 PCB 信息，每行信息均需要带时间、作业编号、进程编号、指令编号等；

进程运行区：位置在右下角，显示正处于运行态进程的 PCB 信息，要带时间、作业编号、进程编号、指令编号等；

内存区：位置在中部，设计二维表的图形化界面可视化呈现内存分配、撤销时的内存管理过程；分配的内存区域用红色，表示出开始地址、结束地址。

进程阻塞区：位置在左下，分解为两个小区域分别显示执行键盘输入、屏幕显示指令以后，进入阻塞态，中断处理以后进行唤醒的信息与过程

(6) 使用 input2 文件夹读取每个指令文件信息；调度运行过程、JCB、PCB 信息变化过程记录保存到 output 文件夹；

1.2.5 优秀要求（成绩等级：A 系列，包括 A、A+）

(1) 利用 4 个线程仿真实现时钟中断及作业并发调度、进程调度、键盘输入及屏幕显示输出中断请求。时钟中断及作业并发调度、进程调度实现代码框架分别写在 ClockInterruptHandlerThread、ProcessSchedulingHandlerThread 文件中，请补充完善代码；键盘输入及屏幕显示输出中断请求，分别使用 2 个单独的新线程程序文件，需要自行设计完成；

(2) 仿真可变分区内存管理，考虑 A、B 两类外部设备，采用静态方式分配外部设备，忽略用户作业的 I/O 操作时间。实现 MMU 地址变换，利用最佳适应算法实现可变分区内存空间分配与回收，注意要考虑内存、外设情况进行作业调度；

(3) 仿真实现内存 A 缓冲区、B 缓冲区管理，分别对应进程执行“键盘输入指令”和“屏幕显示输出指令”，由键盘输入、屏幕显示输出中断请求两个线程分别完成，通过对内存缓冲池管理、PV 同步互斥操作，采用多生产者-多消费者操作；

(4) 多任务作业并发环境下，仿真实现时钟计数、作业请求判别及调度、系统调用、进程创建、进程撤销、进程调度、进程阻塞、进程唤醒、内存分配、内存回收、PV 同步互斥操作、多生产者-多消费者操作等原语；采用三级反馈队列进程调度算法，实现进程就绪态、运行态、阻塞态之间进程切换。

(5) 界面上需要设计 4 个按钮，分别为执行、暂停、保存、实时。

“执行”按钮：按下时系统时钟开始计数并显示，作业调度、进程调度开始工作；

“暂停”按钮：按下时，系统时钟暂时中断、暂停作业调度、进程调度功能；

“保存”按钮：按下时将作业调度、进程调度过程的记录信息保存到 output 文件夹中；

“实时”按钮：按下以后当前时刻有[实时作业请求](#)，实现实时作业指令生成、作业调度、进程调度、进程撤销等操作；实时作业指令包括 20 条，其中 15 条计算类、3 条[键盘输入指令](#)、2 条[屏幕显示输出](#)指令，指令出现顺序自行设计规则。

(6) 界面上划分 8 个区域，分别为[时钟显示区](#)、[作业请求区](#)、[内存区](#)、[进程就绪区](#)、[进程运行区](#)、[进程阻塞区](#)、[A 缓冲区](#)、[B 缓冲区](#)。

时钟显示区：位置在左上角，区域较小，字体为黑色，当仿真程序一旦运行，时钟显示区开始显示按秒计时。

作业请求区：位置在左中，用行形式，每行显示一个作业全部 JCB 信息。

进程就绪区：位置在右中上，分三级区域显示。每级对应三级反馈队列的一级。每一级用行形式显示就绪队列进程 PCB 信息，每行信息均需要带时间、作业编号、进程编号、指令编号等；

进程运行区：位置在右下角，显示正处于运行态进程的 PCB 信息，要带时间、作业编号、进程编号、指令编号等；

内存区：位置在中上部，设计[二维表的图形化界面](#)可视化呈现内存分配、撤销时的内存管理过程；分配的内存区域用红色，表示出开始地址、结束地址。

进程阻塞区：位置在左下，分解为两个小区域分别显示执行键盘输入、屏幕显示指令以后，进入阻塞态，中断处理以后进行唤醒的信息与过程

[A 缓冲区、B 缓冲区](#)：位置在中下部，设计[二维表的图形化界面](#)可视化呈现对缓冲区采用多生产者-多消费者的同步互斥操作；当按下“[执行](#)”按钮对作业开始调度，创建进程就绪区、进程运行区动态显示进程调度过程信息；当执行到“[键盘输入指令](#)”、“[屏幕显示输出指令](#)”时，进程阻塞状态，分别利用 A、B 缓冲区接收或者发送信息到外设；

(7) 使用 [input1 文件夹读取每个指令文件信息](#)；调度运行过程、JCB、PCB 信息变化过程记录保存到 output 文件夹；

2. 基础的仿真设计

2.1 裸机硬件仿真设计

包括 CPU、内存、系统时钟中断、用户内存区、数据输入缓冲区 A、数据输出缓冲区 B、输入输出设备中断、地址变换机构（MMU）等；

以下函数名确定，函数参数、返回值等自行定义完成。

(1) CPU 与寄存器的仿真设计

◆CPU 可抽象为一个类，**名称：CPU**

◆**基础框架代码：src\CPU 文件**，在此文件上需改完善

◆**关键寄存器可抽象为类的属性**，至少包括：程序计数器（PC）、指令寄存器（IR）、状态寄存器（PSW）等，寄存器内容的表示方式自行设计，需要在实践报告中说明。

需要实现 CPU 现场保护、现场恢复操作，封装为 CPU 类的方法，供进程切换、CPU 模式切换方法调用。

方法函数的名称须统一，入口出口参数自行定义；

◆CPU-PRO（）：CPU 现场保护函数；

◆CPU-REC（）：CPU 现场恢复函数；

(2) 系统时钟中断仿真

设计自己的计算机系统时钟，其他中断使用该时钟统一时钟计时；

◆设计一个时钟中断线程，**名称：ClockInterruptHandlerThread**

◆**基础框架代码：src\ClockInterruptHandlerThread 文件**，在此文件上需改完善

-
- ◆该类中设计一个共享属性变量，名称：simulateTime，整型，单位：秒（s）
 - ◆时钟计时函数，名称统一，入口出口参数自行定义：
 - ◆simulateTimePassing（）：通过该函数对 simulateTime 变量计时操作。此变量为临界资源，需要互斥访问；因此，操作该变量时可以用 JAVA 加锁操作：
每 1 秒激活该线程计时，simulateTime+1 操作。
此线程与进程调度线程等其他线程保持同步；
 - ◆**Java 实现系统时钟和多线程交互，实现多线程继承、接口继承、线程同步、线程加锁，可参考本手册 7.2 节实验 2**

（3）作业调度的中断仿真

假设计算机用自己的计时系统，**每 2 秒(s)**查询一次外部是否有新作业的执行请求，在“并发作业请求文件”中判读是否有新作业请求，如果内存、外设 A、B 资源满足时，作业调度运行。

- ◆**基础框架代码：src\ClockInterruptHandlerThread 文件**，在此文件上需改完善
- ◆在时钟中断线程类：**ClockInterruptHandlerThread**，增加作业请求判读函数
- ◆作业请求判读函数，名称：JobRequest（）：
功能：每 2 秒查询一次。利用 simulateTime **变量每计时 2 秒，判断**“并发作业请求文件”是否有新作业请求的时间已到达；如果有，进入**后备队列**；如果没有，空操作；
- ◆作业调度函数，名称：JobDD（）：自行添加函数实现

（4）进程调度的中断仿真

假设计算机 CPU 在 1 秒(s)发生一次时钟硬件中断；可以执行 1 条指令。

- ◆**设计一个进程调度线程类，名称：ProcessSchedulingHandlerThread**
- ◆**基础框架代码：src\ProcessSchedulingHandlerThread 文件**，在此文件上修改完善
 - ◆**通过时钟中断线程激活该线程；**

- ◆进程调度算法函数，根据所选题目等级的调度算法

名称：

SP()：静态优先级法+时间片轮转

MFQ3()：三级反馈队列

- ◆**时间片变量：timeSlice**

功能：根据选题等级设置时间片大小 timeSlice，当 timeSlice=0 时发生进程调度；

（a）申 C、D 成绩完成：

进程切换时间片大小：timeSlice=2 秒，即每个用户进程获得 2s 的 CPU 执行时间，如果 2s 时间片用完，发生进程切换，切换出的用户进程重新进入就绪队列，就绪队列按照优先级排队。在进程已分配的 2 秒时间内，如果收到其他作业请求，不能中断该进程调度。

（b）申 A、B 成绩完成：

采用多级反馈队列调度算法，设计 3 个级别

第 1 级时间片大小：Times=1 秒

第 2 级时间片大小：Times=2 秒

第 3 级时间片大小：Times=4 秒

每个用户进程按照多级反馈队列调度算法进行调度；如果所执行指令有系统调用，则中断，用户进程进入阻塞队列，由与输入输出中断处理线程唤醒被阻塞进程；调度进程线程与输入输出中断处理线程并发

(5) 键盘输入、屏幕显示输出中断仿真

◆申 A、B 成绩实现以下内容：

产生 InputBlockThread 线程类，该线程处理一个 I/O 输入中断指令需要 2 秒：

产生 OutputBlockThread 线程类，该线程处理一个 I/O 输出中断指令需要 3 秒：

ALL_IO（）函数：每个线程均有该函数，无论键盘输入、屏幕显示输出中断请求，如键盘给变量赋值，或者将变量结果显示在屏幕上，均利用该线程处理。假设 OS 内核模块处理 I/O 操作需要 2 秒，之后唤醒进程的阻塞队列。不考虑内存缓冲区操作。I/O 中断处理线程与进程调度、作业调度线程需要并发处理，不能串行处理。

(6) 用户内存区

假设不考虑 OS 内核所占内存空间。

申 A、B、C 成绩需实现以下仿真内容：

◆用户区共 10000B，每个物理块大小 1000B，共 10 个物理块：

◆用位示图实现内存管理。可以假设基础存储单元是 100B，每个物理块占用 10 个基础存储单元。

◆假设用户进程中每条“用户态计算操作语句”类型的指令占用 100B；在进程创建时根据进程指令中计算类型指令个数计算所创建进程所占用内存大小，即：一个进程占用内存大小=该进程中用户态计算操作指令个数*100B。每个进程指令的逻辑地址、物理地址均连续，物理地址的基地址可以不同；

◆地址变换过程(MMU)需要在界面上显示并详细记录过程，保存到 ProcessResults-???-算法名称代号.txt 文件中。??? 表示数字，为每次运行完成所有进程的总分钟数。

例如：测试输入数据是 input1 中的数据文件，用 JTYXJ（静态优先级）算法完成机器调度所有作业，总运行分钟数是 166 秒，保存进程运行输入输出以及调度过程中状态变化、统计数据等所有信息，保存到 ProcessResults-166-LZ.txt 文件，保存到 output1 子文件夹。

(7) 内存缓冲区仿真

◆申 A 成绩需要实现通过输入缓冲区 A、输出缓冲区 B，实现键盘输入数据以及显示器输出显示功能，自行设计实现代码。

◆用于进程 I/O 请求配套的内存缓冲区，输入缓冲区 A、输出缓冲区 B 大小分别为 5 个缓冲区存储单元，每个缓冲区存储单元 100B，A、B 缓冲区大小分别为 500B。假设进程的每条数据输入或者输出指令最大访问 100B，每个进程的每条指令只能分配一个缓冲区存储单元；

◆需要设计 PV 操作原语访问缓冲区

(8) 外设资源

考虑 A、B 两类外部设备，采用静态方式分配外部设备，忽略用户作业的 I/O 操作时间。

A 类外部设备总量：2

B 类外部设备总量：1

2.2 并发的作业请求的设计

◆每个作业相当于用户提交给操作系统运行的可执行程序。本实验考虑作业管理、内存分配回收，进程管理、输入缓冲区、输出缓冲区、进程同步与互斥等功能；

◆创建 input1、input2、input3、input4 子文件夹，保存仿真的作业请求、用户程序指令文件

input1 子文件：申请 A 成绩的同学使用；

input2 子文件：申请 B 成绩的同学使用；

Input3 子文件：申请 C 成绩的同学使用；

Input4 子文件：申请 D 成绩的同学使用；

◆ jobs-input.txt 文件格式固定，由教师统一提供，初始 5 个作业。一行代表一个作业请求，学生测试程序时可以增加新作业条目；

◆ jobs-input.txt 文件内容：

作业序号 (JobsID)

作业请求时间 (InTimes)

优先级 (Priority)

请求 A 资源数量 (Res-A)

请求 B 资源数量 (Res-B)

◆ Priority=0 为不考虑优先级；Priority 数字越大优先权最小，进程调度首先选择优先权小的进程；

◆ InTimes：作业达到时间。以自己的计时系统开始计时，单位是秒。例如：InTimes 的值为 8，是指 8 秒后该作业提出请求；

◆ 作业调用过程如下：仿真程序开始运行时，开始按秒计时（不要用系统时间，用自己的计时器计时）。读取 input 文件夹中作业文件的文件名，一次性读入一个临时数组。**lockInterruptHandlerThread 类每 2 秒(s)查询临时数组**，根据当前计时器时间，通过判断与作业请求时间（单位为秒）中请求时间的匹配程度，判断该时刻是否有作业请求。如果有，加入**作业后备队列**，并将该作业文件内容保存到相应数据结构中。

◆ jobs-input.txt 文件举例：

```
*jobs-input -
文件(F) 编辑(E) 1
1,2,0,1,1
2,2,0,1,0
3,2,0,0,1
4,10,0,0,0
5,10,0,1,0
6,14,0,1,1
7,20,0,1,0
```

2.3 用户程序指令类型的设计

◆ 用户程序指令文件是每个作业要执行的被编译以后的程序指令集合。每个作业的指令类型由本文档给出。每行一条指令，每个作业的指令类型由本文档给出。

(1) 作业程序的结构

作业程序语句有 **4 类**，按照申请成绩等级，分别保存在 input1、input2、input3、input4 子文件夹中。具体包括：

指令编号 (Instruc_ID)

程序语句的类型 (Instruc_State)

(2) 用户程序指令的类型 (Instruc_State)

★ 本试验假设 CPU 执行用户态每条指令，在执行一条机器指令时不可以中断。

★ 指令编号 (Instruc_ID)：作业创建进程以后，进程所执行用户程序段指令序号，从 1 开始计数，该编号为每条指令的逻辑地址，连续编码。

★ 程序语句的类型 (Instruc_State)，包括：

0 表示用户态计算操作指令。执行该类型指令需要运行时间 InRunTimes=1s。

如果 CPU 执行该类型指令时，有优先级高的抢占进程请求，则执行完该指令以后被抢占：

一个进程占用内存大小=该进程中用户态计算操作指令个数*100B

1 表示键盘输入变量指令。发生系统调用，CPU 进行模式切换，运行进程进入阻塞态；值守的键盘操作输入模块接收到输入变量或输出变量内容，InRunTimes=2s后完成输入，产生硬件终端信息号，阻塞队列 1 的队头节点出队，进入就绪队列；

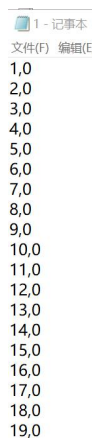
InputBlockThread 类在 2s 以后自动唤醒该进程；

2 表示屏幕显示输出指令。发生系统调用，CPU 进行模式切换，运行进程进入阻塞态；值守的屏幕显示模块输出变量内容，InRunTimes=2s后完成显示，产生硬件终端信息号，阻塞队列 2 的队头节点出队，进入就绪队列；InputBlockThread 类在 2s 以后自动唤醒该进程；

举例：

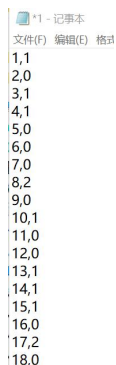
申请 D 成绩的 input4 文件夹中 1 号作业的程序指令保存在 1.txt 文件中。2 号、3 号用户作业指令保存在 2.txt、3.txt 文件中。文件第一行为代号。

1.txt 文件内容如下：



```
1,0
2,0
3,0
4,0
5,0
6,0
7,0
8,0
9,0
10,0
11,0
12,0
13,0
14,0
15,0
16,0
17,0
18,0
19,0
```

申请 B 成绩的 input2 文件夹中 1 号作业的程序指令保存在 1.txt 文件中，内容如下：



```
1,1
2,0
3,1
4,1
5,0
6,0
7,0
8,2
9,0
10,1
11,0
12,0
13,1
14,1
15,1
16,0
17,2
18,0
```

2.4 作业运行及调度详细记录文件的格式

◆作业调度与时钟用同一个线程实现，进程调度模块用单独的线程实现。需要重点注意的是：不是每次发生时钟中断都要进行进程调度。

◆进程调度算法代号：时间片轮转调度算法的代号 RR，静态优先级调度算法的代号 STYXJ，多级反馈队列调度算法代号 DJFK。

◆界面可视化显示作业请求与进程调度过程，需要以文件形式保存。界面显示和文件保存的信息要一致、清晰、完整、可读；

◆本次实验与 input1 或者 input2 或者 input3 或者 input4 中子文件测试数据对应的输出文件夹保存 output 文件夹，文件名称为 ProcessResults-???-算法名称代号.txt 文件。

◆文件读写操作可参考附录 1

◆ProcessResults-???-算法名称代号.txt 文件的格式如下：

(1) 包括 3 段信息，分别为：作业/进程调度事件、消息通信缓冲区处理事件、状态统计信息。其中，作业/进程调度事件、状态统计信息是必须的，其他事件的记录需要根据选题及申请成绩等级来完成。

(2) 进程调度事件信息段的输出信息，每行有时间信息，按照时间顺序，记录发生进程调度事件、进程就绪队列、进程阻塞队列等的状态。

(3) 每段输出信息的关键信息加半角中括号[]。批改时会所有带有中括号的内容进行提取，不加中括号都视为无效信息。

每一行信息的具体格式如下：

时间:[关键操作或状态名称: 作业或进程信息,]

时间需要用仿真计时器的时间；

输出信息格式如下：

作业/进程调度事件：

- 0:[新增作业:作业编号,请求时间,指令数量]
- 1:[创建进程:进程 ID,PCB 内存块始地址,分配内存大小]
- 2:[进入就绪队列:进程 ID:待执行的指令数]
- 3:[运行进程:进程 ID:指令编号,指令类型编号,物理地址, 数据大小]
- 4:[运行进程:进程 ID:指令编号,指令类型编号,物理地址, 数据大小]
- 5:[运行进程:进程 ID:指令编号,指令类型编号,物理地址, 数据大小]
- 6:[阻塞进程:阻塞队列编号,进程 ID 列表]
- 7:[重新进入就绪队列:进程 ID 列表（每个 ID 用/隔开）,对应每个进程对应的剩余没有执行的指令数（每个进程对应的用/隔开）]
- 8:[阻塞进程:阻塞队列编号,进程 ID 列表]
- 9:[CPU 空闲]
- 10:[终止进程 ID]

数据缓冲区处理事件：

- 3:[P 操作/V 操作: 信号量变量名称=具体数值]说明：两个操作选择之一，组合显示
- 4:[拷贝入缓冲区 A: 进程 ID: 指令编号: 所操作的缓冲区编号]
- 5:[拷贝入缓冲区 A: 进程 ID: 指令编号: 所操作的缓冲区编号]
- 6:[P 操作/V 操作: 信号量变量名称=具体数值]说明：两个操作选择之一，组合显示
- 7:[P 操作/V 操作: 信号量变量名称=具体数值]说明：两个操作选择之一，组合显示
- 8:[拷贝出缓冲区 B: 进程 ID: 指令编号: 所操作的缓冲区编号]
- 9:[P 操作/V 操作: 信号量变量名称=具体数值]说明：两个操作选择之一，组合显示
- 10:[缓冲区 A 无进程]

进程状态统计信息：

结束时间:[进程 ID:作业请求时间+进入时间+总运行时间]

结束时间:[进程 ID:作业请求时间+进入时间+总运行时间]

BB1:[阻塞队列 1,键盘输入:进程 ID/进入时间/唤醒时间,进程 ID/进入时间/唤醒时间]

BB2:[阻塞队列 2,屏幕显示:进程 ID/进入时间/唤醒时间,进程 ID/进入时间/唤醒时间]

说明：

在“作业/进程调度事件”中,符号均为半角西文态符号:

时间: 每秒记录一条信息, 两条记录的时间可以相同, 表示并发操作;

关键操作或状态: 包括新增作业、创建进程、第一次进入就绪队列、重新进入就绪队列阻塞队列、**系统调用**、重新进入就绪队列、运行进程、CPU 空闲、终止进程、发送消息等, 输出时请写汉字信息;

记录事件的多少根据选题确定;

作业或进程信息: 包括进程 ID, 指令编号, 指令类型编号等, 信息用逗号隔开;

在“状态统计信息”中: 当每个进程指令全部执行完成以后计算保存。每一行统计一个进程信息;

结束时间: 指进程运行结束后的时间;

2.5 进程控制块 PCB 设计

下面是进程相关 PCB 内容, 参照 Linux task_struct 的数据结构内容设计;

进程编号 (ProID): 值分别为 1, 2, 3, 4, 5, 6, 。。。

进程优先数 (Priority);

进程创建时间 (InTimes);

进程结束时间 (EndTimes);

进程状态 (PSW);

进程运行时间列表 (RunTimes);

进程周转时间统计 (TurnTimes);

进程包含的指令数目 (InstrucNum);

程序计数器信息 (PC);

指令寄存器信息 (IR);

在就绪队列信息列表 (包括: 位置编号 (RqNum)、进入就绪队列时间 (RqTimes));

阻塞队列信息列表 1 (包括: 位置编号 (BqNum1)、进程进入键盘输入阻塞队列时间 (BqTimes1));

阻塞队列信息列表 2 (包括: 位置编号 (BqNum2)、进程进入显示器输出阻塞队列时间 (BqTimes2));

说明：

★系统请求运行的并发进程个数最小值为 5, 可以自行设计最大值。

★进程编号 (ProID): 整数

★进程优先级 (Priority): 整数

★进程创建时间 (InTimes): 由仿真时钟开始计时, 整数, 假设每条指令执行时间 1s;

★进程结束时间 (EndTimes): 显示仿真时钟的时间, 整数;

★进程运行时间列表 (RunTime): 统计记录进程开始运行时间、时长, 时间由仿真时钟提供;

★进程包含的指令数目 (InstrucNum): 可自行扩展;

★PSW: 保存该进程当前状态: 运行、就绪、阻塞;

★指令寄存器信息 (IR): 正在执行的指令编号;

★程序计数器信息 (PC): 下一条将执行的指令编号;

◆需要设计系统空白 PCB 表, **假设系统中进程最大并发数为 12 个**, 可用数组、链表、队列实现。

3 时间节点安排

3.1 第1周：周一晚上 7:00-8:00 线上辅导，讲解课程设计指导手册，布置基础实验 1，学生建立讨论组

3.2 第2周：周一晚上 7:00-8:00 线上辅导，多线程并发交互，布置基础实验 2

3.4 第3周：周一晚上 7:00-8:00 线上辅导，发课设材料模版，分享讲解实验 3，线上答疑，学生编写课设代码

3.5 第4周：周三晚上 7:00-8:00 线上答疑，学生修改课设代码并测试

3.6 第5周，3月17日-21日，全周停课课设，地点：滨江校区实验室（地点另行通知）

3.7 正式提交全部材料时间：

3月20日（周四）提交自己的课设完整材料；

3月21日（周五）提交互测完整材料

3.8 停课5天时间安排：

停课课设期间每日点名，提交每日工作报告，作为平时成绩。具体安排以后续通知为准：

（1）第1-2天：发测试报告模板文件，学生完善程序功能、修改代码测试等；

（2）第3天：完成程序代码测试，撰写自测分析报告，录制测试视频文件；

（3）第4天：完成程序代码测试，完成自测分析报告、录制带讲解的视屏文件等；提交自己参加课设所有正式材料；

（4）第5天：学生相互测试，填写互测报告；提交互测所有正式材料。

（5）因特殊原因不能到校上课的学生，需要提前一周提供学工办、教务办签字的证明，按照课设时间节点提交材料。

（6）第7-12周，完成课设内容进一步测试与检查

（7）第13-16周，完成抽查答辩及成绩统计

（8）本学期课设采用新的自测分析报告模板、互测评价报告模板

4 自测材料提交要求

（1）自测材料提交时间：2025年3月20日（周四）16:00前，只提交一次，过期不收。

（2）要求学生提交完整全面的可执行程序、全套工程文件及代码、自测分析报告、自我演示说明等材料。

◆提交材料要求格式正确性、完整性、讲解清晰，程序能够利用设置断点分段演示

（3）创建“姓名学号-申请成绩”文件夹，根据申请成绩等级拷贝教师提供的、所发布的课设材料中已创建好文件夹。

input1、input2、input3、input4子文件夹中分别是申A、申B、申C、申D成绩等级的输入测试数据：

output子文件夹中分别保存程序运行结果。程序每次运行结果保存到一个新的ProcessResults-???-算法名称代号.txt文件中，该文件不覆盖。

例如，申请B以上成绩的同学，使用input2子文件夹中测试输入数据，程序运行结果保存到output子文件夹中。

(4) “姓名学号-[申请成绩](#)”文件夹的根目录：保存课设程序被编译为 RunProcess.exe 可执行文件、[自测分析报告](#)，并[提交](#)一份“材料阅读使用顺序说明文档”，安装使用说明书、提交材料的阅读顺序及注意事项。

(5) src 子文件夹：工程文件、源程序等，[要求在教师所提供代码框架基础上扩展。扩展的变量、函数、函数输入输出变量有详细的注释，并提供本文件夹每个文件用途的说明视频文件；](#)

(6) test-vidio 子文件夹：根据测试分析模板要求，保存多个[用例代码、测试过程的录屏讲解文件](#)（可以多个）。单个文件大小最好不超过 30M；

(7) west 文件夹：保存使用了第三方程序或框架的库文件等

(8) lab1、lab2 子文件夹：保存所完成的基础性实验完整代码、可执行程序、测试数据与结果

5 课程设计课程联系方式

设计问题联系任课教师或助教，方式可通过 QQ 群或当面交流；

办公地点：滨江校区 A12 栋 A721 室

6 课设基础代码框架的类及函数说明

本次课设需在 src 文件夹下基础代码框架上扩展编程，代码框架说明如下：

控制线程

1. ClockInterruptHandlerThread: 时钟中断线程

变量：

simulateTime: 模拟时间

方法：

simulateTimePassing: 模拟时间流逝

方法：

JobRequest: 检查是否有作业进入，5s 检查一次

2. ProcessSchedulingHandlerThread: 进程调度线程

方法：

RR: 时间片轮转法

SP: 静态优先级法

MFQ: 多级反馈队列

基础数据结构：

3. Instruction: 指令类

变量：

id: 指令序号

state: 指令类型

4. Job: 作业类

变量：

jobId: 作业编号

inTime: 作业进入时间

instructionCount: 作业指令数

5. PCB: 进程控制块（进程类）

变量：

pid: 进程号

pc: 指令计数器

state: 进程状态

仿真硬件

6. CPU: 仿真 CPU

变量:

pc: 指令计数器

ir: 指令寄存器

psw: 程序状态字

registerBackup: 寄存器备份, 用于进程切换时保存寄存器状态

方法:

runProcess: 运行当前进程, 打印相关进程信息

辅助实现程序

7. OSKernel: 操作系统内核, 用于存放队列等相关信息

backupQueue: 后备队列, 用于存放备份的作业

readyQueue: 就绪队列, 用于存放已准备好执行的进程

8. FileUtilis.java: 文件处理类, 根据需求自主设计。

9. SyncManger.java: 同步管理类, 根据选用的进程同步方法, 自主设计。

7 基础实验

以下三个基础实验, 实验步骤及代码说明供参考。供参考代码, 分别在 Lab1、Lab2、Lab3 子文件夹中, 程序需要自行修改调试。

7.1 Lab 1 文件读取

【实验目标】

掌握文件读取的基本操作

理解作业请求的处理流程

实现从文件读取作业请求并加载到系统中

【实验内容】

编写代码实现作业加载的模拟

设计一个简单的作业队列系统

实现作业从模拟外部存储设备加载到模拟操作系统中

【Main 函数测试】

在 Main 函数中, 学生需要完成以下任务:

设置作业文件路径和指令文件夹路径。

调用 loadAllJobsAndInstructions() 方法加载所有作业和指令。

通过 System.out.println() 输出加载的信息, 以验证程序是否按预期工作。

【实验步骤】

Step1 文件基本结构

编写代码从文件读取作业请求, 文件格式为 txt, 参考代码见 Lab 1 文件读取文件夹。

作业输入文件

jobs_input.txt:

1,8,20

2,10,30

3,15,30

4,22,22

5,24,30

作业请求文件：每行表示一个作业请求，包含作业 ID、作业的到达时间和作业包含的指令条数。以 1,8,20 为例，代表 1 号作业在 8 秒时进行请求，作业中的指令条数为 20。

Job 类

```
public class Job {
    private int jobId;           // 作业的唯一标识符
    private int inTime;         // 作业进入系统的时间
    private int instructionCount; // 作业包含的指令数量
    private List<Instruction> instructions; // 与作业关联的指令列表

    public Job(int jobId, int inTime, int instructionCount) {

    }
}
```

作业输入文件

1.txt

1,0

2,0

3,0

...

20,0

作业文件：每行代表一个指令，包含指令 ID、指令类型。以 1,0 为例，代表 1 号指令指令类型为 0。

Instruction 类

```
public class Instruction {
    private int id;           // 指令的唯一标识符
    private int state;        // 指令的状态

    public Instruction(int id, int state) {

    }
}
```

Step2 文件读取和写入

方式 1：使用 BufferedReader 和 BufferedWriter

读取文件：

```
import java.io.*;
import java.util.ArrayList;
import java.util.List;

public class FileReadWriteExample {

    public static List<String> readLines(String filePath) throws IOException
```

```

{
    List<String> lines = new ArrayList<>();
    try (BufferedReader reader = new BufferedReader(new
FileReader(filePath))) {
        String line;
        while ((line = reader.readLine()) != null) {
            lines.add(line);
        }
    }
    return lines;
}

public static void main(String[] args) {
    try {
        List<String> lines = readLines("example.txt");
        for (String line : lines) {
            System.out.println(line);
        }
    } catch (IOException e) {
        e.printStackTrace();
    }
}
}

```

写入文件:

```

import java.io.*;
import java.util.List;

public class FileReadWriteExample {

    public static void writeLines(String filePath, List<String> lines) throws
IOException {
        try (BufferedWriter writer = new BufferedWriter(new
FileWriter(filePath))) {
            for (String line : lines) {
                writer.write(line);
                writer.newLine();
            }
        }
    }

    public static void main(String[] args) {
        List<String> lines = List.of("First line", "Second line", "Third
line");
        try {

```

```

        writeLines("output.txt", lines);
    } catch (IOException e) {
        e.printStackTrace();
    }
}
}

```

方式 2: 使用 Files 类

读取文件

```

import java.nio.file.*;
import java.io.IOException;
import java.util.List;

public class FileReadWriteExample {

    public static List<String> readAllLines(String filePath) throws
IOException {
        return Files.readAllLines(Paths.get(filePath));
    }

    public static void main(String[] args) {
        try {
            List<String> lines = readAllLines("example.txt");
            for (String line : lines) {
                System.out.println(line);
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}

```

写入文件

```

import java.nio.file.*;
import java.io.IOException;
import java.util.List;

public class FileReadWriteExample {

    public static void writeLines(String filePath, List<String> lines) throws
IOException {
        Files.write(Paths.get(filePath), lines);
    }

    public static void main(String[] args) {

```

```

        List<String> lines = List.of("First line", "Second line", "Third
line");
        try {
            writeLines("output.txt", lines);
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}

```

Step3 作业请求线程的实现

JobRequestHandlerThread: 假设计算机每 5 秒(s) 查询一次外部是否有新作业的执行请求，在“并发作业请求文件”中判读是否有新进程请求运行。

类伪代码

```

类 JobRequestHandlerThread 继承自 Thread {

    方法 run() {
        无限循环 {
            调用 handleJobRequests()
        }
    }

    私有方法 handleJobRequests() {
        获取 SyncManager 的锁
        尝试 {
            // 等待作业请求信号
            等待 SyncManager 的 jrtCondition 信号
            调用 processIncomingJob()
            如果当前时间是 5 的倍数 {
                调用 transferJobsToReadyQueue()
            }
            // 通知线程调度线程
            通知所有等待在 pstCondition 上的线程
        } 捕获中断异常 {
            恢复中断状态
            打印异常堆栈
        } 最终 {
            释放 SyncManager 的锁
        }
    }

    私有方法 processIncomingJob() {
        如果 OSKernel 的缓冲区不为空 {
            获取缓冲区队列的第一个作业
            如果作业的进入时间等于当前时间 {

```

```

        从缓冲区移除作业
        将作业加入到后备队列
        打印 "当前时间:[新增作业:作业信息]"
    }
}

私有方法 transferJobsToReadyQueue() {
    当后备队列不为空 {
        获取后备队列的第一个作业
        如果作业的进入时间小于等于当前时间 {
            创建一个新的进程，参数为作业和下一个进程 ID
            从后备队列移除该作业
            将进程加入到就绪队列
            打印 "当前时间:[进入就绪队列:进程信息]"
        } 否则 {
            跳出循环 // 后备队列按到达时间排序，后面的作业还没到时间
        }
    }
}
}

```

伪代码解释

1. run() 方法：该方法是线程的入口，线程启动后会在无限循环中不断调用 handleJobRequests 方法处理作业请求。

2. handleJobRequests() 方法：此方法首先获取 SyncManager 的锁，然后等待作业请求信号。在接收到信号后，处理新到的作业，并根据当前时间判断是否将作业从后备队列转移到就绪队列。最后，通知所有等待在 pstCondition 上的线程。

3. processIncomingJob() 方法：此方法检查 OSKernel 的缓冲区是否有作业请求。如果有并且请求的进入时间与当前时间相同，则将该作业从缓冲区移到后备队列，并打印作业信息。

4. transferJobsToReadyQueue() 方法：此方法遍历后备队列，将所有进入时间小于或等于当前时间的作业移到就绪队列，并创建相应的进程。后备队列按作业到达时间排序，如果某个作业还未到时间，则停止转移作业。

【代码结构说明】

1. FileUtilis.java: 文件读取的基本操作，主程序入口。
2. Instruction.java: 指令类的基本操作内容。
3. Job.java: 作业类的基本操作内容。
4. InstructionLoader.java: 指令读取，将作业和指令进行关联，

7.2 Lab2 系统时钟和多线程交互

【实验目标】

理解系统时钟的设计与实现

掌握多线程之间的交互与同步机制

实现一个能够模拟系统时钟和作业请求处理的多线程程序

【实验内容】

设计并实现一个系统时钟模块

实现多线程交互，模拟作业请求处理

实现线程间的同步与通信

【实验任务】

时钟中断线程（ClockInterruptHandlerThread）：

完成 simulateTimePassing 方法，使得每经过 milliseconds 毫秒，模拟时间增加。

在 run() 方法中正确模拟时间流逝，每秒增加一次模拟时间，并打印输出。

完成时钟中断通知，确保在适当时机通过 SyncManager.pstCondition.signal() 通知进程调度线程。

进程调度线程（ProcessSchedulingHandlerThread）：

完成 run() 方法，确保线程等待时钟中断信号。

实现进程调度逻辑。学生可以选择模拟基于优先级、作业到达时间等进行调度。

完成进程调度后输出“完成进程调度”，并通过 SyncManager.clkCondition.signal() 发出信号通知时钟线程。

同步管理类（SyncManager）：

理解并使用 SyncManager 类中的锁和条件变量进行线程间的同步。

使用 SyncManager.lock.lock() 获取锁，确保线程间同步。

在适当的位置使用 await() 和 signal() 方法来控制线程的阻塞和唤醒。

【Main 函数测试】

在 main 函数中，学生需要完成以下任务：

Main 第一部分为第一次实验的内容，第二部分为线程的控制启动，时钟线程和调度线程的依次执行。

调用 processSchedulingHandlerThread.start() 和 clockInterruptHandlerThread.start() 方法加载所有作业和指令。

通过 System.out.println() 输出加载的信息，以验证线程是否正确执行是否按预期工作。

【实验步骤】

Step1 系统时钟设计

在计算机操作系统中，统一时钟计时是通过系统时钟来实现的。系统时钟是一个定时器，定期触发中断，以便操作系统可以执行一些定时任务，例如进程调度、资源管理和系统监控。在本实验中，我们将设计一个时钟线程类来模拟系统时钟的功能，每秒计时并增加一个计数器。

ClockInterruptThread 类

类 ClockInterruptHandlerThread 继承自 Thread：

```
静态整型 simulationTime = 0
```

```
静态整型 milliseconds = 100
```

```
方法 run()：
```

```
while true：
```

```
    SyncManager.lock.lock()
```

```
    try：
```

```
        SyncManager.jrtCondition.signal()
```

```
        SyncManager.clkCondition.await()
```

```

        simulateTimePassing(milliseconds)
    捕获 InterruptedException 异常 as e:
        e.printStackTrace()
    finally:
        SyncManager.lock.unlock()

    静态方法 getCurrentTime():
        返回 simulationTime

    静态方法 simulateTimePassing(整型 milliseconds):
        try:
            Thread.sleep(milliseconds)
            simulationTime++
        捕获 InterruptedException 异常 as e:
            e.printStackTrace()

```

伪代码解释

1. ClockInterruptHandlerThread 类继承自 Thread 类。它包含两个静态变量 simulationTime 和 milliseconds，分别用于跟踪模拟时间和表示时间模拟的间隔。run 方法是线程的主要方法，在无限循环中持续执行，首先获取锁，然后通知作业请求处理线程，接着等待时钟中断信号，并模拟时间流逝。如果捕获到 InterruptedException 异常，打印异常堆栈跟踪。finally 块确保锁被释放。

2. getCurrentTime 方法返回当前模拟时间。

3. simulateTimePassing 方法模拟指定毫秒数的时间流逝。如果捕获到 InterruptedException 异常，打印异常堆栈跟踪。

Step2 Java 中多线程同步的策略和方法

1. Synchronized 关键字

常见使用场景：

方法同步：适用于需要确保整个方法在同一时间只能被一个线程访问的场景，例如修改共享资源的方法。

代码块同步：适用于只需要同步部分代码块的场景，通过减少锁的粒度来提高性能，例如只在访问共享资源时进行同步。

同步方法

使用 synchronized 关键字可以声明同步方法，这样在同一时间内只有一个线程可以执行该方法。

```

public class SynchronizedExample {
    public synchronized void synchronizedMethod() {
        // 线程安全的代码
    }
}

```

同步代码块

使用 synchronized 关键字可以声明同步代码块，通过指定一个对象作为锁。

```

public class SynchronizedBlockExample {
    private final Object lock = new Object();
}

```

```
public void synchronizedBlockMethod() {
    synchronized (lock) {
        // 线程安全的代码
    }
}
```

2. ReentrantLock 关键字和 Condition

ReentrantLock 是一个可重入锁，提供了比 synchronized 更加灵活的锁机制。

ReentrantLock 常见使用场景：

复杂的同步需求：需要更细粒度的锁控制，例如可中断的锁获取、超时的锁获取和非块结构的锁释放。

实现公平锁：控制线程获取锁的顺序，避免线程饥饿。

尝试获取锁：希望尝试获取锁，但不会无限期等待锁释放的场景。

```
import java.util.concurrent.locks.Lock;
import java.util.concurrent.locks.ReentrantLock;

public class ReentrantLockExample {
    private final Lock lock = new ReentrantLock();

    public void lockMethod() {
        lock.lock();
        try {
            // 线程安全的代码
        } finally {
            lock.unlock();
        }
    }
}
```

与 ReentrantLock 一起使用，Condition 提供了类似 Object 的 wait 和 notify 的功能，但更加灵活。

Condition 常见使用场景：

复杂的线程通信：需要更复杂的等待/通知机制，而不仅仅是简单的 wait/notify，例如多条件变量。

实现生产者-消费者模式：控制生产者和消费者之间的同步，通过条件变量协调生产消费的速度。

等待特定条件：在获取锁的同时等待特定条件的场景，例如某个队列不为空时才能取出元素。

示例：

1. 使用 Condition 实现生产者-消费者模式：

```
import java.util.concurrent.locks.Condition;
import java.util.concurrent.locks.Lock;
```

```

import java.util.concurrent.locks.ReentrantLock;
import java.util.LinkedList;
import java.util.Queue;

public class ConditionExample {
    private final Lock lock = new ReentrantLock();
    private final Condition notEmpty = lock.newCondition();
    private final Condition notFull = lock.newCondition();
    private final Queue<Integer> queue = new LinkedList<>();
    private final int MAX_SIZE = 10;

    public void produce(int value) throws InterruptedException {
        lock.lock();
        try {
            while (queue.size() == MAX_SIZE) {
                notFull.await();
            }
            queue.add(value);
            notEmpty.signal();
        } finally {
            lock.unlock();
        }
    }

    public int consume() throws InterruptedException {
        lock.lock();
        try {
            while (queue.isEmpty()) {
                notEmpty.await();
            }
            int value = queue.poll();
            notFull.signal();
            return value;
        } finally {
            lock.unlock();
        }
    }
}

```

2. 等待特定条件

```

import java.util.concurrent.locks.Condition;
import java.util.concurrent.locks.Lock;
import java.util.concurrent.locks.ReentrantLock;

public class ConditionWaitExample {

```

```

private final Lock lock = new ReentrantLock();
private final Condition condition = lock.newCondition();
private boolean conditionMet = false;

public void awaitCondition() throws InterruptedException {
    lock.lock();
    try {
        while (!conditionMet) {
            condition.await();
        }
        // 执行条件满足后的操作
    } finally {
        lock.unlock();
    }
}

public void signalCondition() {
    lock.lock();
    try {
        conditionMet = true;
        condition.signal();
    } finally {
        lock.unlock();
    }
}
}

```

3. Semaphore 信号量

常见使用场景：

限制访问资源的线程数量：控制同时访问共享资源的线程数量，例如数据库连接池。

实现多资源的管理：协调多个线程对多个共享资源的访问，例如限制某一时刻只能有固定数量的线程执行某个任务。

实现某些类型的生产者-消费者问题：允许指定数量的生产者和消费者进行同步工作。

示例：

1. 限制访问资源的线程数量：

```

import java.util.concurrent.Semaphore;

public class SemaphoreExample {
    private final Semaphore semaphore = new Semaphore(3); // 允许 3 个线程同时访问
    private int count = 0;

    public void accessResource() {
        try {
            semaphore.acquire(); // 获取许可

```

```

        try {
            // 访问共享资源的代码
            count++;
            System.out.println("Resource accessed by thread: " +
Thread.currentThread().getName() + " | Count: " + count);
            Thread.sleep(1000); // 模拟资源访问的耗时操作
        } finally {
            semaphore.release(); // 释放许可
        }
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
}

public static void main(String[] args) {
    SemaphoreExample example = new SemaphoreExample();
    for (int i = 0; i < 10; i++) {
        new Thread(example::accessResource).start();
    }
}
}

```

2. 实现多资源的管理：

```

import java.util.concurrent.Semaphore;

public class MultiResourceSemaphoreExample {
    private final Semaphore semaphore;

    public MultiResourceSemaphoreExample(int resourceCount) {
        this.semaphore = new Semaphore(resourceCount);
    }

    public void useResource(int resourceId) {
        try {
            semaphore.acquire(); // 获取许可
            try {
                // 使用资源的代码
                System.out.println("Resource " + resourceId + " used by thread:
" + Thread.currentThread().getName());
                Thread.sleep(1000); // 模拟资源使用的耗时操作
            } finally {
                semaphore.release(); // 释放许可
            }
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}

```

```

    }
}

    public static void main(String[] args) {
        MultiResourceSemaphoreExample example = new
MultiResourceSemaphoreExample(5); // 允许同时使用的资源数
        for (int i = 0; i < 10; i++) {
            final int resourceId = i % 5;
            new Thread(() -> example.useResource(resourceId)).start();
        }
    }
}

```

3. 实现生产者-消费者模式：

```

import java.util.LinkedList;
import java.util.Queue;
import java.util.concurrent.Semaphore;

public class ProducerConsumerSemaphoreExample {
    private final Queue<Integer> queue = new LinkedList<>();
    private final Semaphore notFull;
    private final Semaphore notEmpty = new Semaphore(0);
    private final Semaphore mutex = new Semaphore(1);
    private final int MAX_SIZE = 5;

    public ProducerConsumerSemaphoreExample() {
        this.notFull = new Semaphore(MAX_SIZE);
    }

    public void produce(int item) throws InterruptedException {
        notFull.acquire(); // 等待有空余空间
        mutex.acquire(); // 获取互斥锁
        try {
            queue.add(item);
            System.out.println("Produced: " + item);
        } finally {
            mutex.release(); // 释放互斥锁
            notEmpty.release(); // 通知有新元素可消费
        }
    }

    public int consume() throws InterruptedException {
        notEmpty.acquire(); // 等待有可消费的元素
        mutex.acquire(); // 获取互斥锁
        try {

```

```

        int item = queue.poll();
        System.out.println("Consumed: " + item);
        return item;
    } finally {
        mutex.release(); // 释放互斥锁
        notFull.release(); // 通知有空余空间
    }
}

public static void main(String[] args) {
    ProducerConsumerSemaphoreExample example = new
ProducerConsumerSemaphoreExample();

    // 生产者线程
    Thread producer = new Thread() -> {
        for (int i = 0; i < 10; i++) {
            try {
                example.produce(i);
                Thread.sleep(500); // 模拟生产耗时
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    });

    // 消费者线程
    Thread consumer = new Thread() -> {
        for (int i = 0; i < 10; i++) {
            try {
                example.consume();
                Thread.sleep(1000); // 模拟消费耗时
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    });

    producer.start();
    consumer.start();
}
}

```

【代码结构】

1. ClockInterruptHandlerThread.java: 模拟时钟线程，时间流逝。
2. JobThread.java: 模拟作业请求线程，每五秒钟请求一次。

-
3. SyncManager.java: 用于存储同步变量。
 4. Test.java: 用于启动线程, 主程序入口。

7.3 Lab 3 进程调度

【实验目标】

理解进程调度的基本概念与算法。

学习如何实现常见的进程调度算法, 如先来先服务 (FCFS)、最短作业优先 (SJF) 和轮转调度 (Round Robin)。

实现一个简单的进程调度模拟系统。

【实验内容】

1. 初始化作业队列:

维护一个作业队列, 模拟不同作业的到达时间和执行时间。每个作业都有一个唯一标识符、到达时间、优先级等。

2. 时钟中断线程 (ClockInterruptHandlerThread):

每秒钟增加模拟时间, 并通过 SyncManager.pstCondition.signal() 通知进程调度线程。

根据当前时间判断是否有新的作业到达

时钟中断线程每次增加时间并通知进程调度线程, 进程调度线程根据调度算法选择作业。

3. 进程调度线程 (ProcessSchedulingHandlerThread):

接收时钟中断信号后, 判断当前是否需要执行作业或者调度新作业。

调度策略: 基于选择的调度算法, 判断哪个作业应当执行或调度。

4. 作业的执行与调度:

根据调度算法和作业的状态, 进程调度线程会执行作业。

如果作业的时间片用完, 或者作业执行完毕, 则需要将作业从当前队列中移除或者重新排队。

5. 作业执行的状态更新:

每个作业有执行时间, 执行后需要更新其状态。执行完成的作业应当进入终止队列或重新调度。

【实验任务】

进程调度线程 (ProcessSchedulingHandlerThread):

实现进程调度算法 (FCFS、RR、SJF 或优先级调度等), 根据调度规则选择作业并执行。

确保调度和执行逻辑与时钟线程交互良好, 避免死锁或同步问题。

作业队列的管理:

学生需要实现作业队列的管理, 作业的状态更新 (等待、执行、完成)。

调度算法的实现:

选择并实现调度算法, 确保作业按优先级或其他规则进行调度。

输出调度过程和作业执行信息。

【实验步骤】

Step1 Process 类的设计

包含属性: 进程 ID、到达时间、执行时间、剩余时间等。

实现构造函数、获取属性的方法和 toString() 方法。

Process 类

```
package jobmanager;
```

类 `Process` 继承自 `Job`:

属性:

整型 `pid` // 进程 ID
 整型 `pc` // 程序计数器 (Program Counter)
 整型 `state` // 进程状态

方法:

构造方法 `Process(jobId, inTime, instructionCount, pid)`:
 调用父类的构造方法初始化 `jobId`, `inTime`, `instructionCount`
 初始化 `pid` 为参数 `pid`
 初始化 `pc` 为 0
 初始化 `state` 为 0 // 假设 0 表示就绪状态

构造方法 `Process(job, pid)`:
 调用父类的构造方法初始化 `jobId`, `inTime`, `instructionCount`
 初始化 `pid` 为参数 `pid`
 初始化 `pc` 为 0
 初始化 `state` 为 0 // 默认状态
 调用 `setInstructions(job.getInstructions())` 复制指令列表

伪代码解释

`Process` 类继承自 `Job` 类, 包含三个属性: 进程 ID (`pid`), 程序计数器 (`pc`), 和进程状态 (`state`)。它有两个构造方法。

`Process` 构造方法使用 `jobId`、`inTime`、`instructionCount` 和 `pid` 参数初始化 `Process` 对象, 调用父类 `Job` 的构造方法来初始化作业 ID、进入时间和指令数量, 然后将 `pid` 初始化为传入的参数 `pid`, 将 `pc` 初始化为 0, 将 `state` 初始化为 0, 假设 0 表示就绪状态。

`Process` 构造方法使用 `Job` 对象和 `pid` 参数来初始化 `Process` 对象, 调用父类 `Job` 的构造方法来初始化作业 ID、进入时间和指令数量, 然后将 `pid` 初始化为传入的参数 `pid`, 将 `pc` 初始化为 0, 将 `state` 初始化为 0 作为默认状态, 并调用 `setInstructions(job.getInstructions())` 方法复制指令列表。

Step2 调度算法实现

先来先服务 (FCFS) 调度算法伪代码

类 FCFS:

```
方法 schedule(processList):
    按照到达时间排序 processList
    currentTime = 0
    对于每个 process 在 processList 中:
        如果 currentTime 小于 process 的到达时间:
            currentTime = process 的到达时间
        打印 "进程 process 的 ID 在 currentTime 开始"
        currentTime += process 的执行时间
        打印 "进程 process 的 ID 在 currentTime 结束"
```

FCFS 类的 `schedule` 方法接受一个进程列表 `processList` 作为参数, 首先根据进程的到达时间对列表进行排序。然后初始化当前时间 `currentTime` 为 0。对于 `processList` 中的每

个进程，检查当前时间是否小于进程的到达时间，如果是，则将当前时间更新为进程的到达时间。然后打印进程的开始时间和结束时间，并将当前时间增加进程的执行时间。

4. 先来先服务（FCFS）调度算法伪代码

```
类 RoundRobin:
    属性:
        整型 timeQuantum // 时间片长度

    构造方法 RoundRobin(timeQuantum):
        初始化 timeQuantum 为参数 timeQuantum

    方法 schedule(processList):
        使用队列 queue 存储 processList 中的进程
        currentTime = 0
        当 queue 不为空:
            process = queue 的第一个元素
            如果 process 的到达时间小于等于 currentTime:
                executeTime = 最小值(timeQuantum, process 的剩余时间)
                打印 "进程 process 的 ID 在 currentTime 开始"
                currentTime += executeTime
                process 的剩余时间 减少 executeTime
                如果 process 的剩余时间 > 0:
                    将 process 添加到 queue 的尾部
            否则:
                打印 "进程 process 的 ID 在 currentTime 结束"
                将 process 从 queue 中移除
        否则:
            currentTime++
```

① 初始化和准备阶段:

RoundRobin 类有一个属性 timeQuantum，表示时间片的长度。在 RoundRobin 类的构造方法中，使用传入的 timeQuantum 参数初始化该属性。

schedule 方法接受一个进程列表 processList 作为参数，并使用队列 queue 存储 processList 中的进程。初始化当前时间 currentTime 为 0。

② 进程调度循环:

当 queue 不为空时，获取队列中的第一个进程 process。

检查该进程的到达时间是否小于等于当前时间。如果是，则计算该进程的执行时间 executeTime，其值为时间片长度 timeQuantum 和进程剩余时间中的最小值。

打印进程的开始时间，然后将当前时间增加执行时间，并减少进程的剩余时间。

③ 处理剩余时间和时间片轮转

如果进程的剩余时间大于 0，则将进程重新添加到队列的尾部；否则，打印进程的结束时间，并将进程从队列中移除。

如果进程的到达时间大于当前时间，则当前时间增加 1。这一步确保系统时间的推进，即使当前没有进程可以执行。

5. FCFS 多级反馈队列（Multilevel Feedback Queue）调度算法伪代码

```
类 MultilevelFeedbackQueue:
```

属性:

整型 `timeQuantum` // 基础时间片长度

列表 `queueList` // 多级队列列表

整型 `levels` // 多级队列的数量

构造方法 `MultilevelFeedbackQueue(timeQuantum, levels):`

初始化 `timeQuantum` 为参数 `timeQuantum`

初始化 `levels` 为参数 `levels`

初始化 `queueList` 为长度为 `levels` 的空队列列表

方法 `schedule(processList):`

`currentTime = 0`

将 `processList` 中的所有进程按照到达时间排序并添加到 `queueList[0]`

当 `queueList` 中的任一队列不为空:

对于每个 `level` 从 0 到 `levels-1`:

如果 `queueList[level]` 不为空:

`process = queueList[level]` 的第一个元素

如果 `process` 的到达时间小于等于 `currentTime`:

`executeTime = 最小值(timeQuantum * (level + 1), process 剩`

余时间)

打印 "进程 `process` 的 ID 在 `currentTime` 开始于等级

`level`"

`currentTime += executeTime`

`process` 的剩余时间 减少 `executeTime`

如果 `process` 的剩余时间 `> 0`:

如果 `level < levels - 1`:

将 `process` 添加到 `queueList[level + 1]` 的尾部

否则:

将 `process` 重新添加到 `queueList[level]` 的尾部

否则:

打印 "进程 `process` 的 ID 在 `currentTime` 结束"

将 `process` 从 `queueList[level]` 中移除

否则:

`currentTime++`

跳出内层循环

① 初始化和准备阶段:

`MultilevelFeedbackQueue` 类有三个属性: `timeQuantum` 表示基础时间片长度, `queueList` `levels` 是多级队列的数量。在 `MultilevelFeedbackQueue` 类的构造方法中, 使用传入的 `timeQuantum` 和 `levels` 参数初始化这些属性, 并将 `queueList` 初始化为长度为 `levels` 的空队列列表。

② 进程调度准备:

`schedule` 方法接受一个进程列表 `processList` 作为参数。

初始化当前时间 `currentTime` 为 0。

将 `processList` 中的所有进程按照到达时间排序，并添加到 `queueList[0]` 中，即最高优先级队列。

③ 进程调度循环：

当 `queueList` 中的任一队列不为空时，循环处理进程。

对于每个等级 `level` 从 0 到 `levels - 1`，检查对应的队列 `queueList[level]` 是否为空。

如果 `queueList[level]` 不为空，获取队列中的第一个进程 `process`。

检查该进程的到达时间是否小于等于当前时间。如果是，则计算该进程的执行时间 `executeTime`，其值为 `timeQuantum * (level + 1)` 和进程剩余时间中的最小值。

打印进程的开始时间和所在等级，然后将当前时间增加执行时间，并减少进程的剩余时间。

如果进程的剩余时间大于 0，则根据当前等级决定将其重新添加到队列中。如果当前等级小于 `levels - 1`，则将进程添加到下一等级的队列 `queueList[level + 1]`；否则，将进程重新添加到当前等级的队列 `queueList[level]` 的尾部。

如果进程的剩余时间等于 0，则打印进程的结束时间，并将其从当前队列中移除。

如果 `process` 的到达时间大于当前时间，则增加当前时间 `currentTime++`。

处理完一个进程后，跳出内层循环，继续处理下一个进程。

【使用说明】

根据 Lab1 和 Lab2 的内容，将 Lab3 的代码进行适当调整，即可组成可运行的基础程序，最终实现：文件读取，模拟时钟，作业请求和进程调度的基本功能。